Authored by: Arpit Raj, LNMIIT Jaipur
Date: Sunday, October 25, 2026
© 2026 Arpit Raj. All rights reserved.

C++ Advanced Concepts

Casting, final, & Multiple Inheritance

Upcasting vs Downcasting

Upcasting

- -> converting a derived class pointer to a base class pointer

Eg

```
Dog d;
Animal *ptr = &d;
```

this is allowed implicitly because every child class inherits from base class.

- # upcasting does not change the actual object, it changes how we refer to that object.

Downcasting

- -> converts a base class pointer into a derived class pointer

Eg

```
Dog * dogptr = dynamic_cast<Dog*>(baseptr);
// where Animal * baseptr = &d
```

this will work because `baseptr` points to actual dog object.
but if `Animal a; Animal * baseptr = &a;` then in that case `dogptr` is a `nullptr` because downcast fails.

dynamic_cast vs static_cast

```
Animal *p = &dog;
dynamic_cast<Dog*>(p);
```

- checks the actual type at runtime, and is a safe failure handling

```
static_cast<Dog*>(p);
```

- it converts without a runtime type check.
- so even if `*p = &animal`, it will compile but using it as a `Dog*` would be invalid.

> 💡 **Requirement for Dynamic Cast**
> For a runtime checked downcast using `dynamic_cast`, the base class must be **Polymorphic** (i.e. atleast one virtual function), usually a virtual destructor is used.

> 🚨 **CRITICAL: WHY STATIC_CAST IS INVALID HERE**
>
> Static cast is invalid because if `*p = &animal`, the compiler allocates enough memory to hold members of animal only, but a Dog object has more memory requirements hence **Memory Mismatch**.
>
> The program will attempt to write to memory outside boundaries of the Base object which can corrupt adjacent variables or cause a **segment fault**.

The final Keyword

- -> used to prevent further overriding of a virtual function
- -> used to prevent further inheritance from a class

Eg

```
class Dog : public Animal {
public: void sound() final { }
};
// and animal also has sound which is virtual
```

then

```
class Pup : public Dog {
public: void sound() { } // -> will give a compile error
};
```

Pup can inherit from Dog but overriding of sound function forbidden.

similarly: `class Dog final : public Animal { }`
then

```
class Pup : public Dog {
} // -> this is not allowed as no one can inherit from Dog
// public Animal inheritance is allowed
```

Ambiguity in Multiple Inheritance

Q

```
class A { public: virtual void show() { } };
class B { public: virtual void show() { } };
class C : public A, public B { };

C obj;
obj.show();
```

- -> Virtual enables runtime dispatch, but doesnt tell compiler which base function was intended, hence ambiguity compile error.
- sol: use `obj.A::show()`
- # One derived function can override both base function
  - so if inside class C -> `void show() override`, will change both Vtable and override both base functions.

CONCEPTUAL SUMMARY: RESOLVING AMBIGUITIES

- # in Diamond problem we used `virtual` inheritance.
- # to solve runtime dispatch problems we used `virtual` function.

Virtual inheritance resolves duplicate shared base subobject in diamond hierarchy.